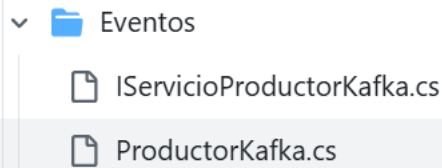


Mi código Kafka:



ProductorKafka.cs :

```
1    using Confluent.Kafka; // me lo reconoce/autocompleta post-hacer dotnet add pac
2    using System.Text.Json;
3    using ReservasApi.Modelos; // para poder poner ": IServicioProductorKafa" sin e
4
5    namespace ReservasApi.Eventos // o ReservasApi.Services o ReservasApi.Messages
6    {
7        public class ProductorKafka : IServicioProductorKafka // KafkaProducerServi
8        { // clase ProductorKafka debe implementar el miembro de la interfaz, dicho
9            // Le defino dos propiedades privadas de solo lectura a esta clase:
10            private readonly IProducer<Null, string> _productor;
11            private readonly string _topic = "reservas.creadas";
12
13            // Defino constructor no parametrizado de esta clase:
14            public ProductorKafka()
15            {
16                var config = new ProducerConfig
17                {
18                    BootstrapServers = "kafka:9092" // mi broker address, el cual p
19                }; // cambio localhost por kafka-broker
20
21                _productor = new ProducerBuilder<Null, string>(config).Build();
22            }
23
```

SIGUE EN NEXT PAGE

```
24 // Ahora sí, implemento el miembro de la interfaz o sea su método:
25 ✓ public async Task PublicarReservaCreadaAsync(Reserva reserva) // producer method
26 {
27     // Serialize using System.Text.Json, esto se escribe/hace diferente si empleam
28     var mensaje = JsonSerializer.Serialize(new
29     {
30         reserva.ReservaId,
31         reserva.Dni,
32         reserva.InstalacionId,
33         reserva.FechaHora,
34         reserva.NumeroAsistentes
35     });
36
37     // Log before publishing (esto lo añado en el proceso de intentar solucionar e
38     Console.WriteLine($"Publicando reserva {reserva.ReservaId} to topic {_topic} c
39
40
41     try // esto lo añado en el proceso de intentar solucionar el problema de que e
42     {
43         // Produce the message to Kafka
44         var resultado = await _productor.ProduceAsync(
45             _topic,
46             new Message<Null, string> { Value = mensaje }
47         );
48
```

TERMINA ESTE FILE EN NEXT PAGE

```

48
49         //Log success
50         Console.WriteLine($"Kafka OK => {resultado.TopicPartitionOffset}");
51     }
52     catch (Exception ex)
53     {
54         // Log any errors:
55         Console.WriteLine($"Error publicando reserva {reserva.ReservaId}: {ex.Message}");
56     }
57 } // final del body de public async Task PublicarReservaCreadaAsync
58
59 } // final de la clase ProductorKafka.
60 }

```

La interfaz que el productor implementa :

[reservas-backend](#) / [ReservasApi](#) / [Eventos](#) / [IServicioProductorKafka.cs](#) 



lagoPol Añado el código kafka a mi backend.

dbcc474 · 2 d

Code

Blame

Raw



```

1     using ReservasApi.Modelos;
2
3     namespace ReservasApi.Eventos
4     { // esta interfaz la va a implementar mi clase ProductorKafka.
5         public interface IServicioProductorKafka // IKafkaProducerService
6         {
7             Task PublicarReservaCreadaAsync(Reserva reserva); // en la implementación precedo esto de public async
8         }
9     }

```

PÁGINA SIGUIENTE EL **RESERVAS CONTROLLER POST API RESERVAS** (porque incluye código Kafka)

NEXT PAGE

```

30 // POST: api/reservas
31 // CreateReserva(Reserva nuevareserva)
32 // The following is the "create" (una nueva reserva) operation. Create reserva.
33 [HttpPost] // matches POST /api/reservas
34 public async Task<ActionResult<ReservaLeerDto>> CrearReserva(ReservaCrearDto dto) // (Reserva nuevareserva)
35 { // the body of the request must contain a JSON representation of a Reserva (de la que quieres crear, supongo).
36     // comienzo del código propio del DTO:
37     // Validate that the user and installation exist :
38     var usuario = await _contexto.Usuarios.FindAsync(dto.Dni); // Usuarios es un DbSet en mi AppDbContext.cs
39     var instalacion = await _contexto.Instalaciones.FindAsync(dto.InstalacionId); // Instalaciones es un DbSet en mi dbcontext
40
41     if (usuario == null || instalacion == null) // validamos that the user and installation exist.
42     {
43         return BadRequest("Usuario o Instalación NO encontrados.");
44     }
45
46     // Create the new booking and link navigation properties (para que 'usuario' e 'instalacion' -navigation properties del modelo
47     var nuevareserva = new Reserva // each time you create a new reservation object in memory inside your controller, that instance
48     {
49         Dni = dto.Dni, InstalacionId = dto.InstalacionId, FechaHora = dto.FechaHora, NumeroAsistentes = dto.NumeroAsistentes,
50
51         // Attach navigation properties (so that EF Core also attaches the Usuario and Instalacion entities based on fks 'dni'
52         Usuario = usuario, // we're populating the existing nav props with the related entities ('usuario' and 'instalacion')
53         Instalacion = instalacion, // if you don't assign these nav props before saving, then when you later query the record,
54     }; // explicitly assigning nav props affects writing/saving, whereas .Include(...) affects reading/querying. I should include
55
56     // creates and saves a new Reserva en otras palabras Add and save:
57     _contexto.Reservas.Add(nuevareserva); // EFCore adds it (la nueva reserva) to the database (Add + SaveChangesAsync()).
58     await _contexto.SaveChangesAsync(); // after SaveChangesAsync(), the navigation properties are available and populated in memory
59     // ese SaveChangesAsync() saves the new booking, después de esto podremos poner la línea de código que publica el evento kafka
60
61     // Map to DTO for response (doesn't affect DB or Kafka):
62     var reservaDto = new ReservaLeerDto
63     {
64         ReservaId = nuevareserva.ReservaId,
65         FechaHora = nuevareserva.FechaHora,
66         NumeroAsistentes = nuevareserva.NumeroAsistentes,
67         Dni = nuevareserva.Dni,
68         InstalacionId = nuevareserva.InstalacionId,
69
70         Usuario = new UsuarioDto
71         {
72             Dni = usuario.Dni,
73             NombreCompleto = usuario.NombreCompleto
74         },
75
76         Instalacion = new InstalacionDto
77         {
78             InstalacionId = instalacion.InstalacionId,
79             Localizacion = instalacion.Localizacion
80         }
81     };
82

```

sigue en next page

```

83         // Placing Kafka publishing after the mapping to DTO is perfectly fine.
84         // Publish Kafka event (un valid placement de esta línea de código es después del "map to DTO" code -en especial debe ir después del await _contexto_SaveChanges
85         await _productorKafka.PublicarReservaCreadaAsync(nuevareserva); // awaits the message to ensure it's successfully sent before returning 201 Created.
86         // la anterior línea es reemplazable por la siguiente:
87         // _ = _productorKafka.PublicarReservaCreadaAsync(nuevareserva); esto es la versión "fire-and-forget" of the Kafka call, se usa if...
88         //...if you don't want to block the HTTP response while Kafka is publishing
89
90         // Return Created (201)
91         return CreatedAtAction(nameof(GetReservas), new { id = nuevareserva.ReservaId }, reservaDto); // antes del 13 de nov aquí ponía nameof(GetReserva)
92     } // returns '201 Created' with the new object (la nueva reserva) and a Location header pointing to /api/reservas/{id}
93

```

Comentarios en líneas 83 y 84:

// Placing Kafka publishing after the mapping to DTO is perfectly fine.

// Publish Kafka event (un valid placement de esta línea de código es después del "map to DTO" code -en especial debe ir después del await _contexto_SaveChangesAsync()- pero antes del return CreatedAtAction i.e. before returning the DTO)

En Program.cs:

using ReservasApi.Eventos; // puse esta línea al ponerme con "Register the Kafka producer in DI (Dependency Injection)"

```

28     // Register the Kafka producer in DI, para ello añado esta línea antes de var app = builder.Build() y después de var builder = WebApplication.CreateBuilder(args):
29     builder.Services.AddSingleton<IServicioProductorKafka, ProductorKafka>(); // ProductorKafka es una clase creada por mí que implementa la interfaz IServicioProductorKafka
30
31     var app = builder.Build(); // ya viene por defecto.

```

docker-compose.yml sobre Kafka (i.e. final de mi docker-compose.yml):

```

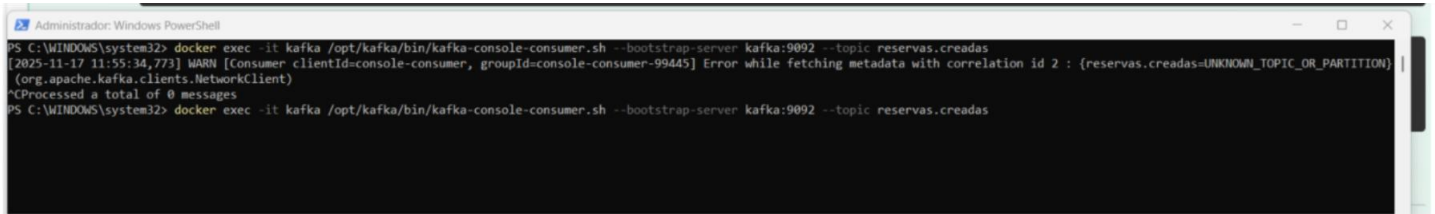
33     kafka:
34         image: apache/kafka:3.8.0
35         container_name: kafka
36         ports:
37             - "9092:9092"
38         environment:
39             # Basic KRaft settings
40             KAFKA_NODE_ID: 1
41             KAFKA_PROCESS_ROLES: broker,controller
42             KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
43             # Listeners
44             KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
45             KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
46             KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
47             KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
48             # Log directories for KRaft metadata
49             KAFKA_LOG_DIRS: /var/lib/kafka/data
50             # Auto topic creation
51             KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
52             # Cluster ID (must be explicitly provided for KRaft), a 16-32 character alphanumeric string, me lo invento yo.
53             CLUSTER_ID: "NlV3PRBMYxyMYHRMFN3Wl"
54         volumes:
55             - kafka_data:/var/lib/kafka/data
56
57     volumes:
58         data_sql: # SQL Server persistence
59         kafka_data: # Kafka persistence

```

Ese Docker-compose.yml te debería crear el contenedor kafka-broker (yo le llamé ‘kafka’ a secas, pero es el kafka-broker) cuando hagas el **docker-compose up --build**.

NEXT PAGE (ya termino con la chapa)

Tmb necesitas tener corriendo el consumidor, el cual puede correr en el propio Kafka-broker container o bien en su propio container (en el del propio consumidor); ambas opciones las consigues en la PowerShell de Windows :



```
Administrador: Windows PowerShell
PS C:\WINDOWS\system32> docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka:9092 --topic reservas.creadas
[2025-11-17 11:55:34,773] WARN [Consumer clientId=console-consumer, groupId=console-consumer-99445] Error while fetching metadata with correlation id 2 : {reservas.creadas=UNKNOWN_TOPIC_OR_PARTITION}
(org.apache.kafka.clients.NetworkClient)
^CProcessed a total of 0 messages
PS C:\WINDOWS\system32> docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka:9092 --topic reservas.creadas
```

El primer consumidor lo hice antes de POST (creador del topic) VS el segundo consumidor después de hacer post (i.e. después de crear el topic debido a KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true" en mi docker-compose.yml), el tema es que no imprime el json message.

Y si cancelo ese segundo consumidor y arranco el del container, éste tmb reconoce el topic i.e. no da el error de Unknown topic:



```
Powershell:
docker-run--rm--network-reservas-app_default--it-apache/kafka:3.8.0
/opt/kafka/bin/kafka-console-consumer.sh
--bootstrap-server-kafka:9092--topic-reservas.creadas--from-beginning
```

Al hacer un post api reservas en swagger mientras tienes corriendo el consumidor (y el Kafka-broker container y el backend container y el sqlserver container), debería aparecerte automáticamente un output en la terminal del consumidor. **A mí esto es lo que no me aparece.**